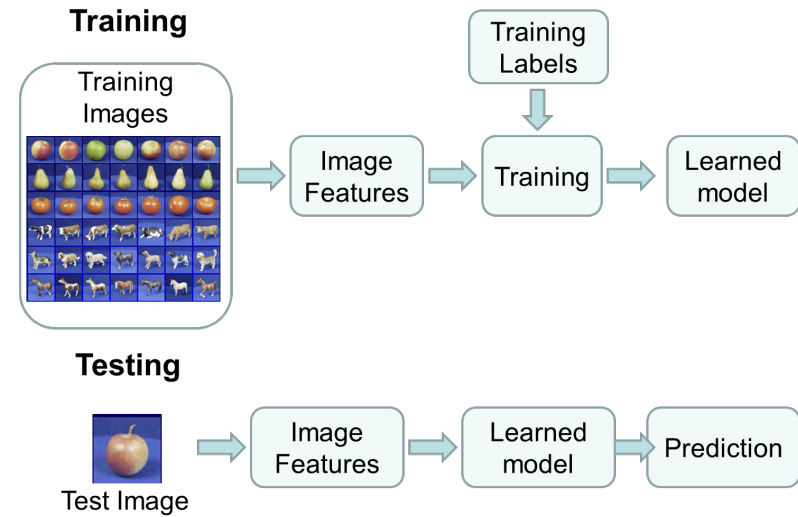


Types of Learning

	<i>Supervised Learning</i>	<i>Unsupervised Learning</i>
<i>Discrete</i>	classification or categorization	clustering
<i>Continuous</i>	regression	dimensionality reduction



Feature Representation

- In context of machine learning, a **feature** is any piece of information which is relevant for solving the computational task related to a certain application.
 - For example, in image processing, a feature describes the content of an image – typically about whether a certain region of the image has certain properties.



LENA and its Canny edge feature

More features include:

- Points,
- Edges,
- Regions...

What is a “Good” Feature?

- Feature representation is a core issue in machine learning – find real “signals” from many “noises”.
- For many years, a lot of manual feature extraction methods have been invented.
- Major criteria:
 - Completeness of signals;
 - Distortion of noises;
 - Feasibility in implementation;
 - Interpretability to users.

[Feature \(computer vision\)](#)
[- Wikipedia](#)

Feature detection

Edge detection

Canny · Deriche · Differential · Sobel · Prewitt ·
Roberts cross

Corner detection

Harris operator · Shi and Tomasi ·
Level curve curvature ·
Hessian feature strength measures · SUSAN ·
FAST

Blob detection

Laplacian of Gaussian (LoG) ·
Difference of Gaussians (DoG) ·
Determinant of Hessian (DoH) ·
Maximally stable extremal regions · PCBR

Ridge detection

Hough transform

Hough transform ·
Generalized Hough transform

Structure tensor

Structure tensor · Generalized structure tensor

Affine invariant feature detection

Affine shape adaptation · Harris affine ·
Hessian affine

Feature description

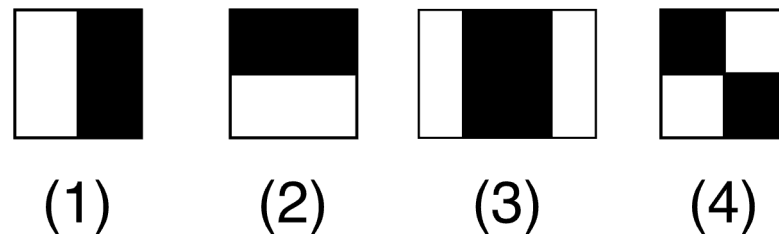
SIFT · SURF · GLOH · HOG

Scale space

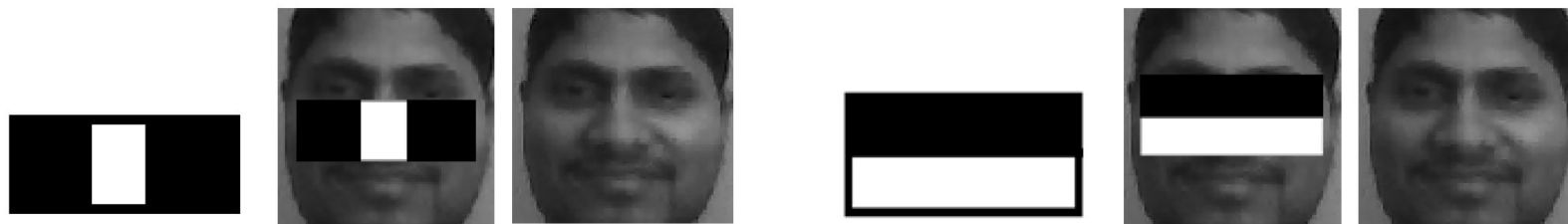
Scale-space axioms · Implementation details ·
Pyramids

Example: Haar-like Features

- Haar-like features are digital image features used in object recognition.



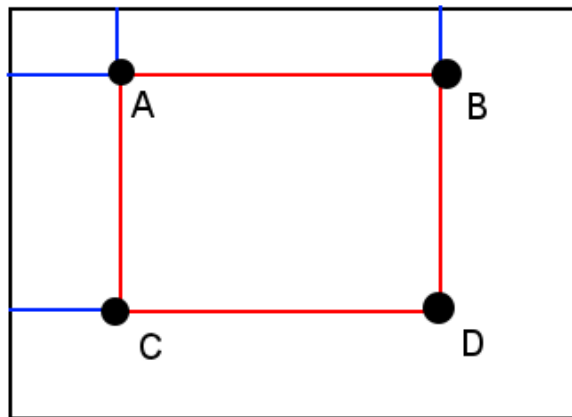
Typical Haar-like filters



Apply individual filters to an image (potentially containing a human face) and generate the feature maps.

Example: Viola-Jones Object Detection

- By cascading Haar-like features with a set of classifiers in the Boosting scheme, the face detection problem was successfully solved in 2000's.



$$\text{Sum} = D - B - C + A$$

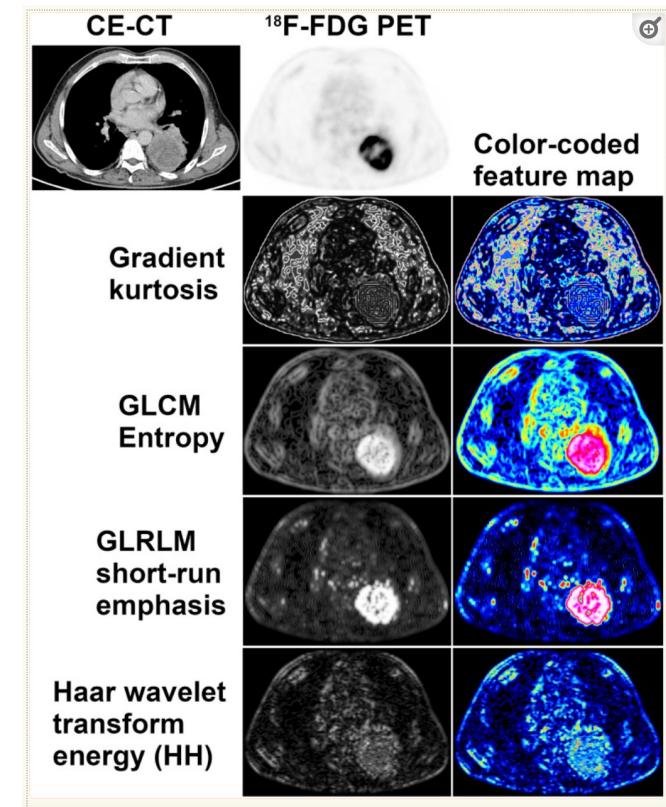
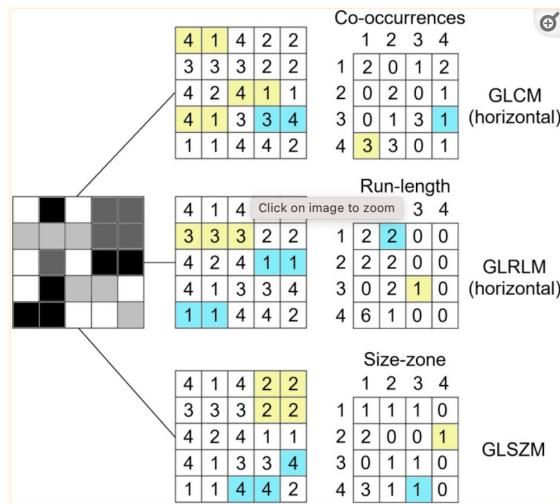
Fast computation of Haar-like features



Digitalization of cameras in 1990-2000's

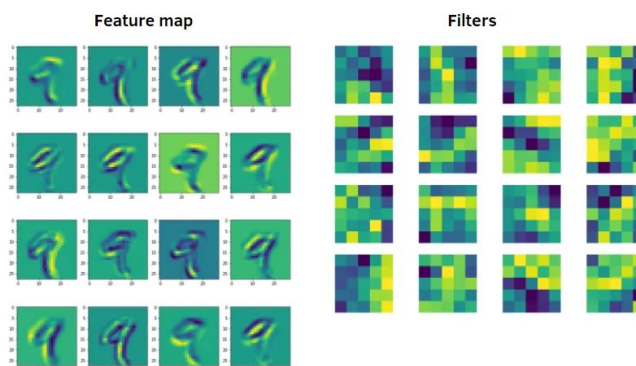
Example: Radiomics Features

- Radiomic features capture tissue and lesion characteristics such as heterogeneity and shape, which be used for clinical problem solving.



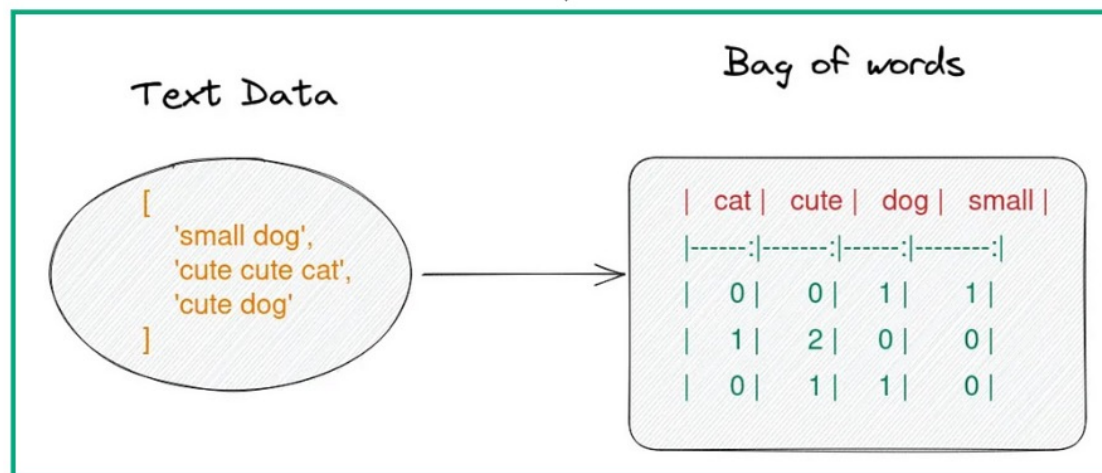
Feature Representation in Vision

- Deep-learning-based features:
 - Deep Convolutional Neural Networks (CNNs) can be trained to extract a set of features that are learned from the task-specific data while minimizing the need for handcrafted features.
 - These features are learned via training and can be used for different applications such as object detection and image classification.



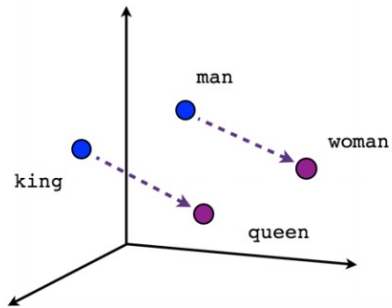
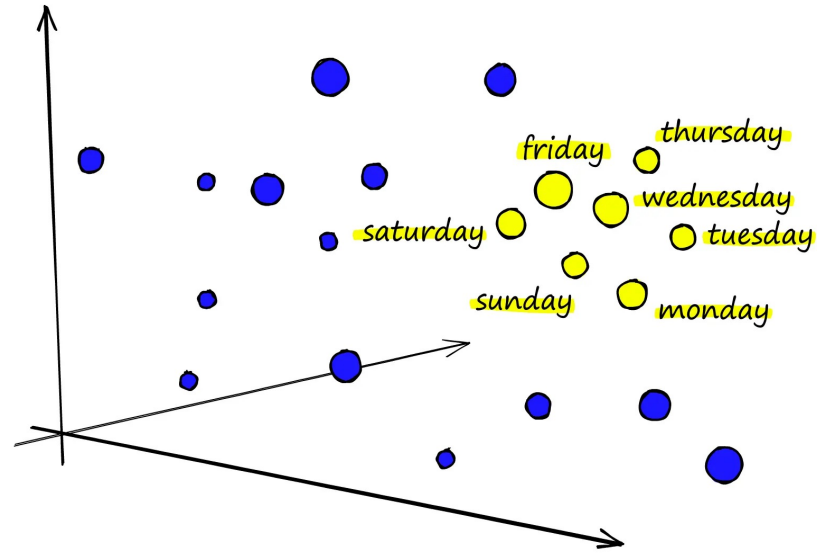
Feature Representation in NLP

- Bag of Words (BoW):
 - In BoW approach, documents are represented as a set of words or tokens, irrespective of their order or context.
 - Each word is represented as a feature, and its frequency within a document is used as its value.

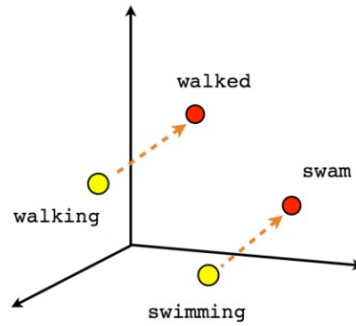


Feature Representation in NLP

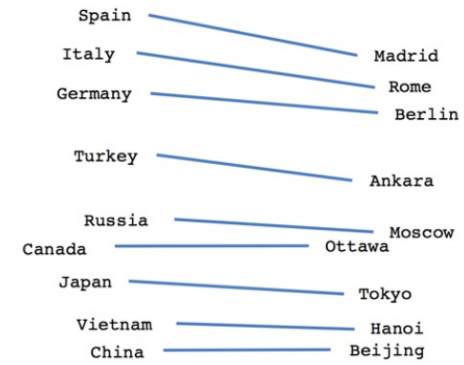
- Word Embeddings:
 - Word embeddings represent words as dense vectors in a continuous vector space where similar words are mapped to nearby points.
 - These embeddings are usually learned from large corpora using neural network architectures such as Word2Vec, GloVe, or FastText.
 - Word embeddings capture semantic relationships between words to some extent.



Male-Female



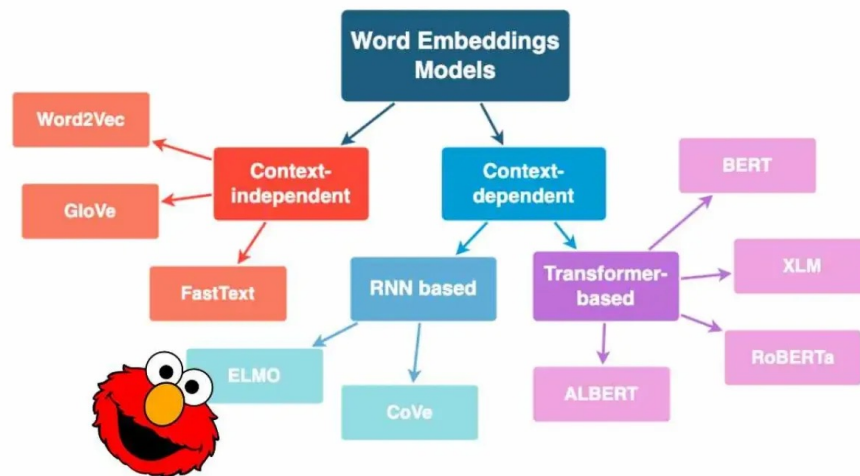
Verb tense



Country-Capital

Feature Representation in NLP

- Contextualized Embeddings:
 - Contextualized word embeddings, such as ELMo, GPT, and BERT, capture the meaning of a word based on its context within a sentence.
 - Unlike traditional word embeddings, these embeddings generate a unique representation for each occurrence of a word, considering its surrounding words.

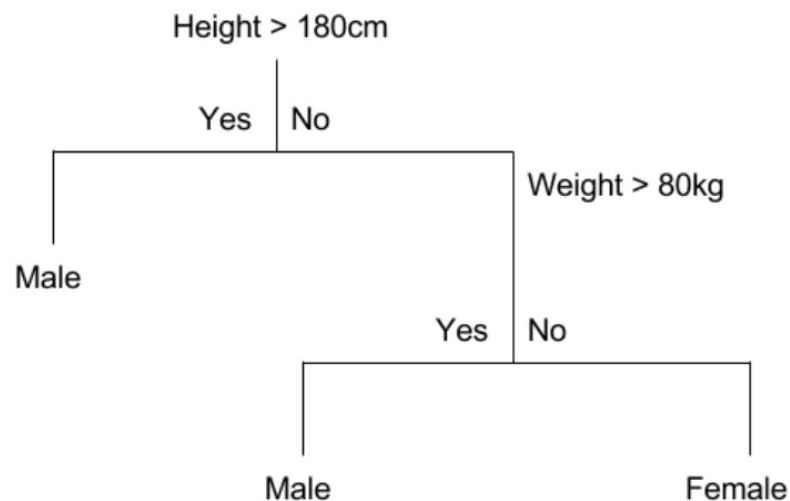


Types of Learning

	<i>Supervised Learning</i>	<i>Unsupervised Learning</i>
<i>Discrete</i>	classification or categorization	clustering
<i>Continuous</i>	regression	dimensionality reduction

Decision tree

- Decision tree learning is a method commonly used in data mining.
 - The goal is to create a model that predicts the value of a target variable based on several input variables or features.



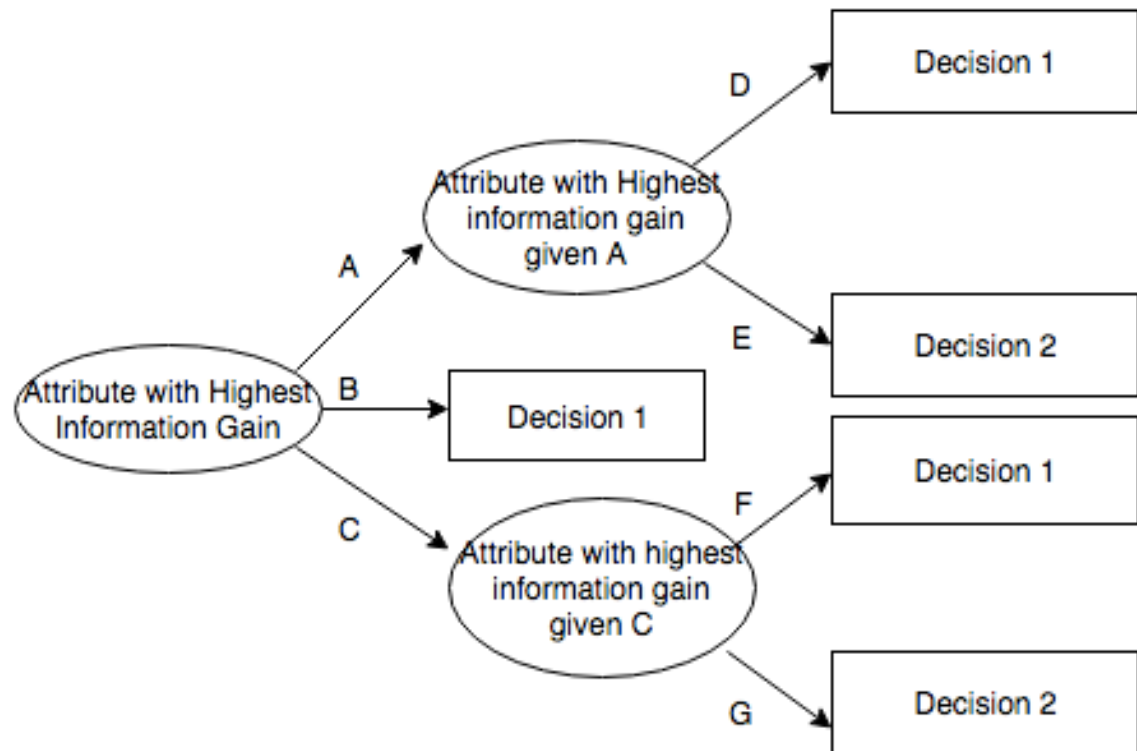
- A classification tree is an algorithm where the target variable is fixed or categorical.
- The algorithm is then used to identify the “class” within which a target variable would most likely fall.

How to Build a Decision Tree?

- ID3 algorithm (Iterative Dichotomiser 3)
- C4.5 – an extension of ID3 and ranking #1 in the *Top 10 Algorithms in Data Mining* pre-eminent paper (<http://www.cs.umd.edu/~samir/498/10Algorithms-08.pdf>)
- CART (Classification and Regression Tree)

How to Build a Decision Tree?

- Root Node: represents the entire dataset.
- Internal Nodes: performs splits or decisions on the data based on a specific feature and threshold.
- Leaf Nodes: provides the output prediction, either a class label (for classification) or a continuous value (for regression).



How to Build a Decision Tree?

- **Selecting the Best Split:** The algorithm searches the dataset to find the best feature and threshold to split the data. The “best split” is the one that results in the most homogenous sub-nodes.
- **Splitting the Data:** The data is split into subsets using the best feature and threshold. This process is recursively applied to each subset.
- **Stopping Criteria:** The recursion is stopped when a predefined condition is met (e.g., all leaf nodes are pure, the number of instances in a node is fewer than a specified number, or the depth of the tree reaches a maximum threshold).
- **Pruning:** Post-processing step to trim down the tree and remove nodes that do not provide significant power in predicting the target variable. This helps to prevent overfitting.

Splitting Criteria

- For classification, we can use the **information gain** (or Gini index which is similar).
 - Information gain is defined as the **reduction in information entropy** after a dataset is split on a feature.
- For regression, it is similar to classification as we can use reduction of variance to choose the feature and threshold for splitting.

Information Entropy

- In information theory, the entropy of a random variable is the average level of "uncertainty" inherent to the variable's possible outcomes.
 - The concept of information entropy was introduced by Shannon in 1948.



Claude Shannon

$$H(x) = - \sum_x P(x) \cdot \log P(x)$$
$$= \sum_x P(x) \cdot \log \left(\frac{1}{P(x)} \right)$$

The prob. of event x WHAT IS THIS?

Information Entropy

- If an event is highly uncertain, it contains a lot of information when it occurs.
 - A fair six-sided die has a higher entropy than a biased one that always lands on the same number.
- Conversely, if an event is very predictable, it contains little information.
 - If you were certain a coin would land heads, its entropy would be zero.



Information Entropy

- The entropy $H(X)$ of a discrete random variable X with possible outcomes $\{x_1, x_2, \dots, x_n\}$ and a probability mass function $P(X) = \{p_1, p_2, \dots, p_n\}$ is given by the formula:

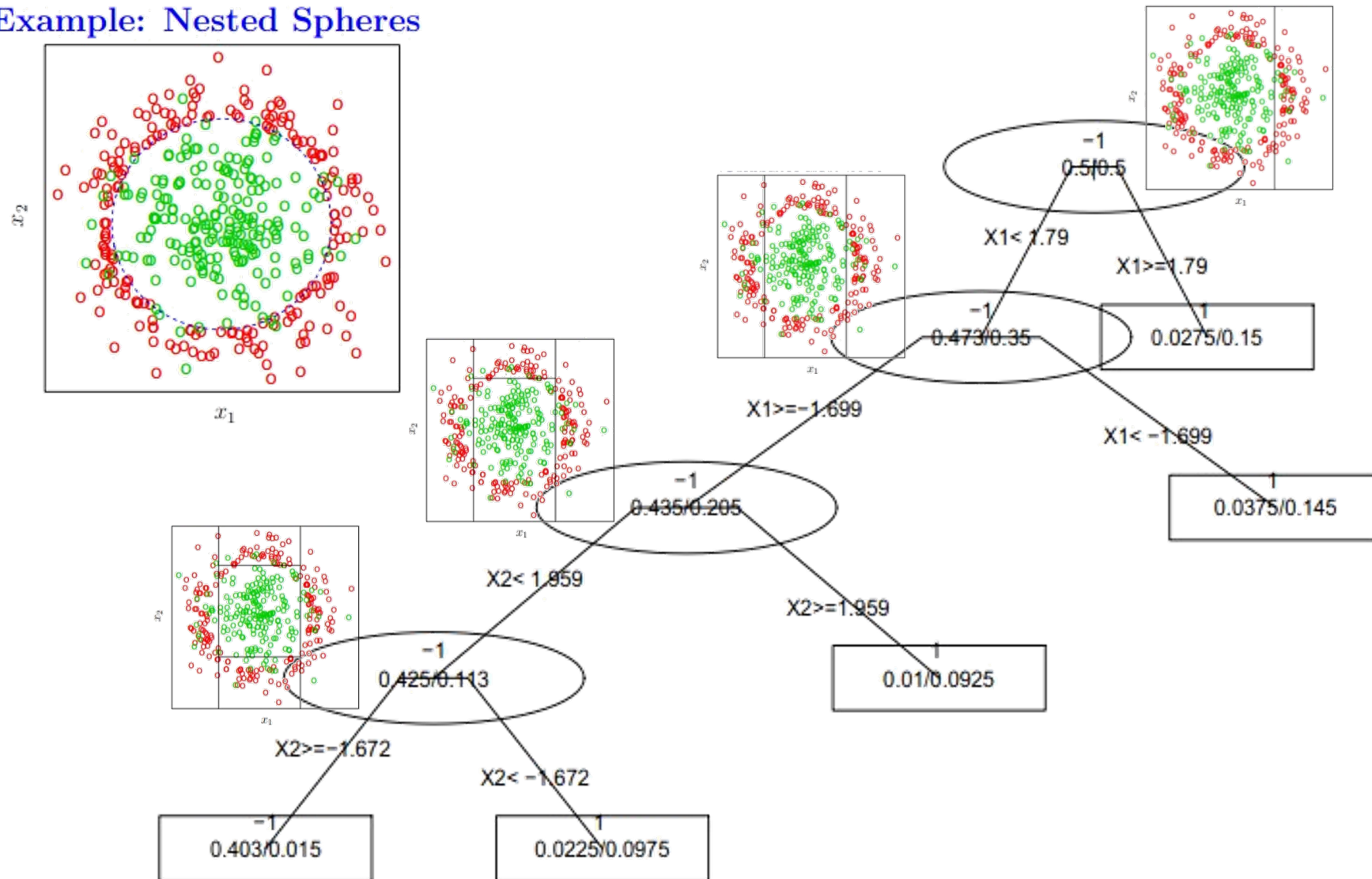
$$H(X) = - \sum_{i=1}^n p(x_i) \log p(x_i)$$



Example: A biased coin with probabilities $P(H)=0.7$ and $P(T)=0.3$.

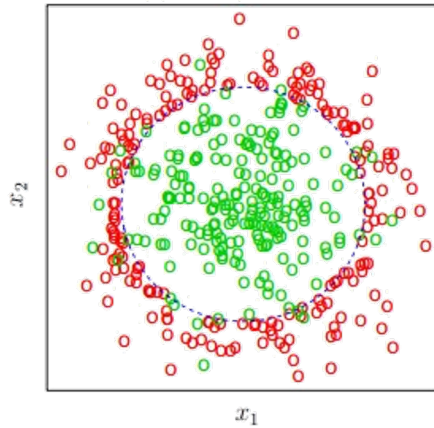
$$\rightarrow H(X) = -(0.7 \log 0.7 + 0.3 \log 0.3) \approx 0.881 \text{ bits}$$

Example: Nested Spheres

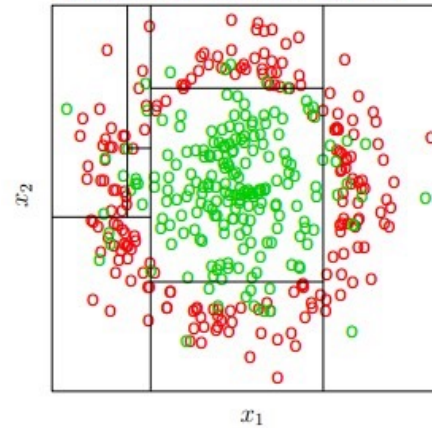


Decision Boundary of a Tree

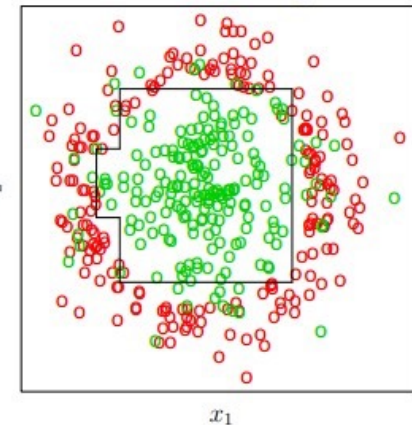
Example: Nested Spheres



Classification Tree



Decision Boundary: Tree

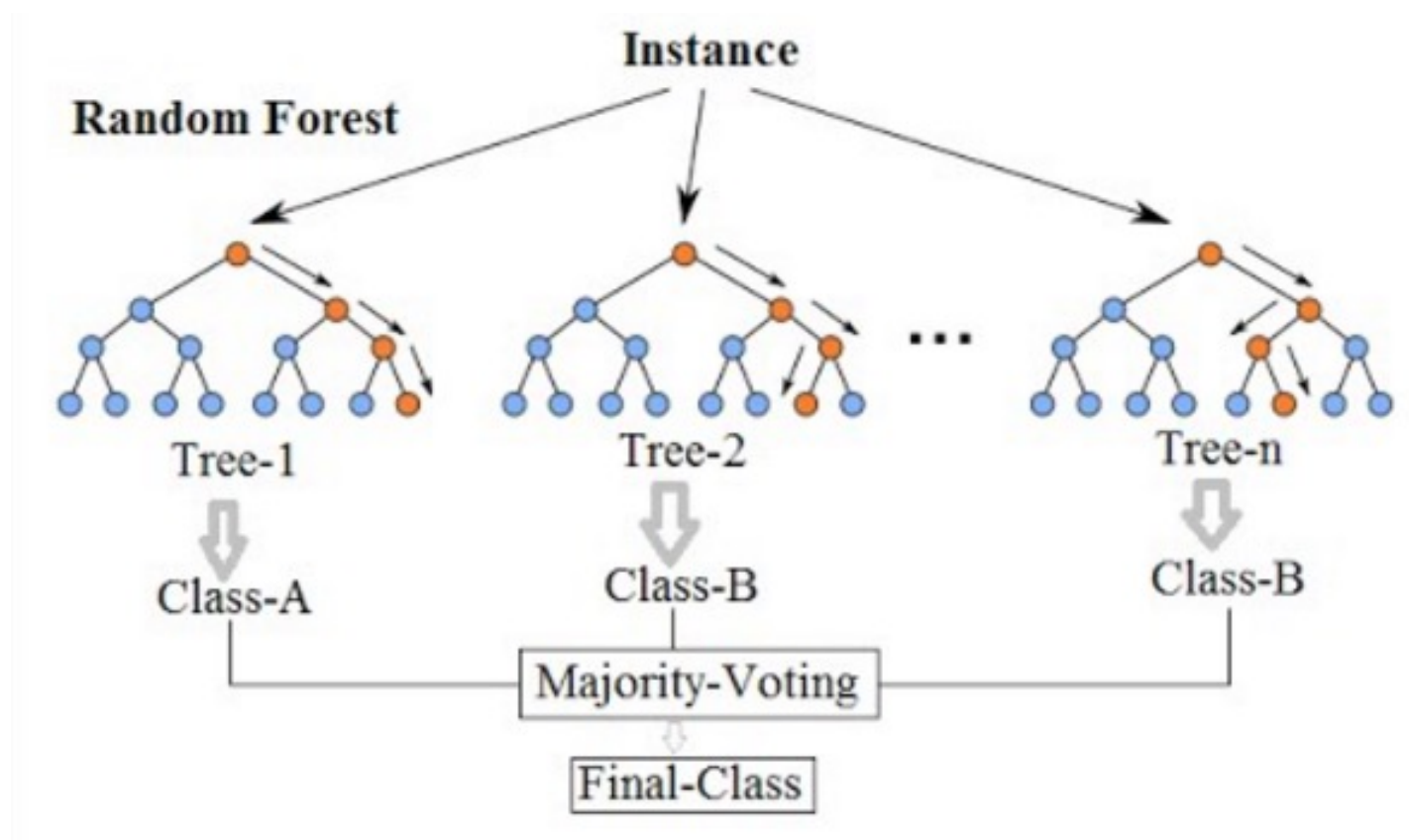


How can we get an even better decision boundary (like a circle)?

Random Forest

- Random forest is an ensemble learning method that operates by constructing a multitude of decision trees at training time.
 - For classification task, the output of the random forest is the class selected by most trees.
 - For regression task, the mean or average prediction of the individual trees is returned.
- The aggregation of several decision trees helps to reduce the variance and avoid overfitting, which can be a problem with individual decision trees.

Random Forest

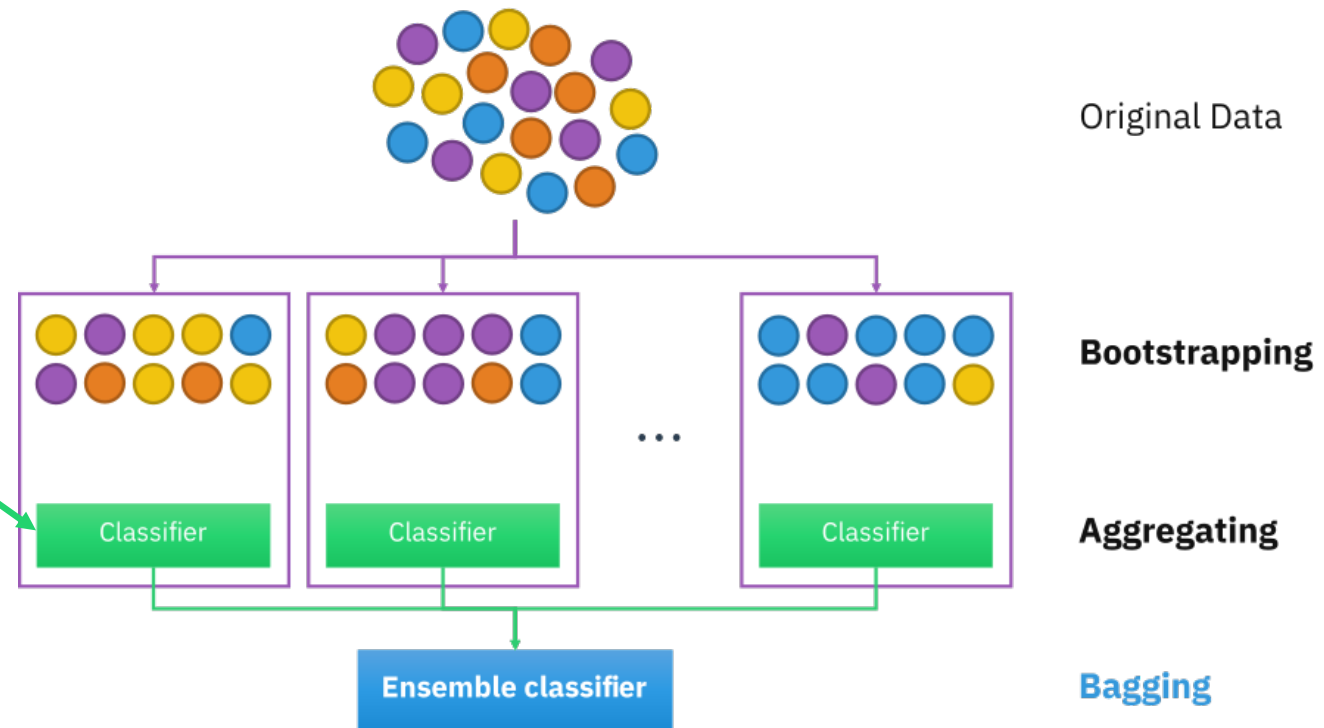


Bagging

- **Bootstrap aggregating**, also called **Bagging**, is an ensemble learning meta-algorithm designed to improve the stability and accuracy of machine learning.
- **Bootstrap** is a statistical method that involves **sampling with replacement** from a dataset.
 - A bootstrap sample is created by randomly selecting instances from the original dataset with replacement until the sample size is equal to the original dataset size.
 - An instance can be selected multiple times in a single bootstrap sample, or it may not be selected at all.
 - Each bootstrap sample can be used to train a separate model (e.g., a tree), which has high diversity due to bootstrap.

Bagging

- At each node, **select a random subset of features**.
- Determine the best split based on the selected features.
- Recursively split the nodes until a stopping criterion is met (e.g., maximum depth, minimum samples per leaf, etc.).



Ensemble Learning

- **Ensemble learning** is a powerful machine learning paradigm where multiple models are combined to solve a particular problem and improve the overall performance compared to individual models.
 - **Bagging**: Multiple models are trained independently (or parallelly) using different samples of the training data.
 - **Boosting**: Train models sequentially, where each new model focuses on correcting the errors made by the previous models.
 - **Stacking**: Train multiple base models (level-0 models) and then train another model (meta-model or level-1 model) to best combine their predictions.

Boosting

- Boosting: converts **weak** learners to **strong** ones sequentially.
 - In boosting, each weak learner is trained on a subset of the training data, and the algorithm assigns higher weights to misclassified samples in each iteration.
 - The subsequent weak learners, which are trained on the data that are more likely to be misclassified by the previous learners, are then combined using a weighted sum to produce the final model.
 - This process continues until a predefined stopping condition is met, such as when the accuracy of the model reaches a threshold or the maximum number of iterations is reached.

Boosting



AdaBoost

- AdaBoost, which stands for Adaptive Boosting, is one of the earliest and most influential boosting algorithms.
- Key Concepts of AdaBoost
 - Weak Learner: Often a simple model like a decision stump (a decision tree with only one split).
 - Sequential Training: Each new learner attempts to correct the errors of its predecessor by giving more weight to the misclassified instances.
 - Weighted Data: During training, data points that were misclassified by previous learners are given higher weights, making them more important for the next learner.
 - Combination of Learners: The final prediction is a weighted combination of the predictions of all the weak learners. Each learner's contribution to the final prediction is based on its accuracy, with more accurate learners having a greater influence.

[Read additional tutorial document!](#)

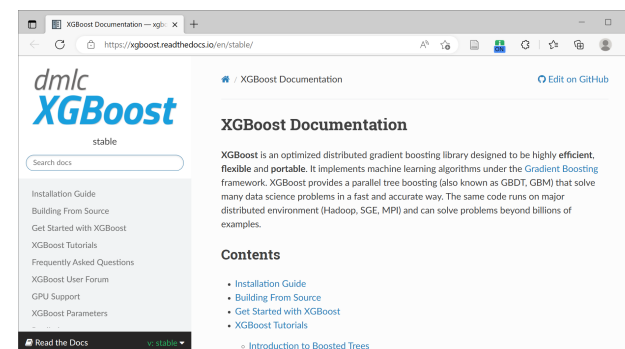
Gradient Boosting Machine

- Gradient Boosting Machine (GBM) is a powerful and widely-used machine learning technique for regression and classification tasks.
 - Gradient boosting builds an ensemble of weak learners, typically decision trees, by training them sequentially such that each new model corrects the errors made by the previous models.
 - The term “gradient” refers to the optimization technique used to minimize the loss function.
 - Popular implementations: XGBoost (eXtreme Gradient Boosting), LightGBM (Light Gradient Boosting Machine).

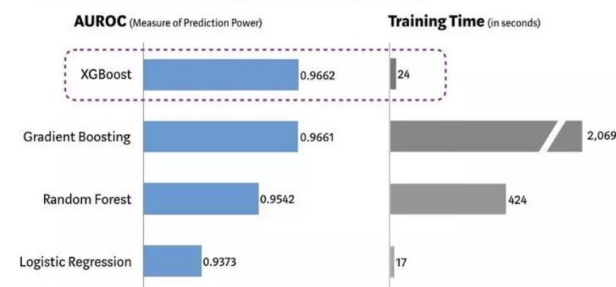
Read additional tutorial document!

XGBoost

- XGBoost (eXtreme Gradient Boosting) is an open-source software library which provides a regularizing gradient boosting framework for C++, Java, Python, R, Julia, Perl, and Scala.
- It works on Linux, Microsoft Windows, and macOS.
- It aims to provide a "Scalable, Portable and Distributed Gradient Boosting (GBM, GBRT, GBDT) Library".
- It runs on a single machine, as well as the distributed processing frameworks Apache Hadoop, Apache Spark, Apache Flink, and Dask.



Performance Comparison using SKLearn's 'Make_Classification' Dataset
(5 Fold Cross Validation, 1MM randomly generated data sample, 20 features)



XGBoost

- Speed and Performance:
 - Known for its high execution speed and performance optimization techniques.
 - It leverages parallel processing and efficient data structures.
- Model Accuracy:
 - Includes regularization to reduce overfitting and improve accuracy.
- Scalability:
 - Can handle very large datasets efficiently, both in terms of rows and columns.
- Community and Support:
 - Large community support with extensive documentation and numerous tutorials available. Widely used in data science competitions (e.g., Kaggle).

Open Resources

- <https://scikit-learn.org/stable/>
- <https://xgboost.readthedocs.io/en/latest/index.html>